

BuzzCart Project Report

Index

1. Abstract
2. Introduction
3. Problem Statement
 - 3.1 Untrusted Product Reviews
 - 3.2 Poor Product Understanding
4. Objectives
5. Proposed Solution
6. System Architecture
 - 6.1 High-Level Architecture
 - 6.2 Frontend Layer
 - 6.3 Backend Layer
 - 6.4 Database Layer
 - 6.5 Storage Layer
 - 6.6 Cache Layer
 - 6.7 AI RAG Layer
7. Database Design Summary
8. Main Modules and Features
 - 8.1 Authentication and User Profiles
 - 8.2 Product Management
 - 8.3 Shopping Cart and Checkout
 - 8.4 Social Feed
 - 8.5 Videos and Reels
 - 8.6 Messaging
 - 8.7 Search and Discovery
 - 8.8 Media and Document Uploads
9. Core Innovation 1: Trusted Reviews From Known People
 - 9.1 Verified Purchase Enforcement
 - 9.2 Verified Purchase Badge
 - 9.3 Socially Ranked Reviews
 - 9.4 Helpful Votes
 - 9.5 Moderation and Privacy
 - 9.6 Buyer Visibility

10. Core Innovation 2: RAG for Better Product Understanding 10.1 Why It Matters 10.2 Document Pipeline 10.3 Retrieval Strategy 10.4 Grounded Answers 10.5 Product-Specific Scope 10.6 Business Value 11. How the System Works for End Users 11.1 Buyer Workflow 11.2 Seller Workflow 11.3 Review Workflow 11.4 Product QA Workflow 12. Major Selling Points of the Project 13. Technical Strengths 14. Limitations 15. Future Enhancements 16. Conclusion

1. Abstract

BuzzCart is a social commerce platform that combines short-form content, ecommerce, trusted reviews, and an AI-powered product assistant into one system. The platform is built with a Flutter frontend, a Go backend, PostgreSQL for transactional data, Redis for caching, Firebase Storage for media and document hosting, and a Python-based Retrieval-Augmented Generation (RAG) service for document-grounded product question answering.

The project focuses on two major real-world problems. First, online product reviews are often hard to trust because users cannot easily tell whether the reviewer is a real buyer or someone socially relevant to them. Second, buyers struggle to understand product details from long brochures, manuals, or specification sheets. BuzzCart addresses these problems by surfacing reviews from known or connected people first, highlighting verified purchases, and providing a RAG assistant that answers product questions directly from uploaded product documents.

This project demonstrates a practical full-stack architecture for modern social shopping experiences, where product discovery, trust, and product understanding are tightly integrated.

2. Introduction

Digital commerce platforms usually separate product catalogs, social engagement, customer reviews, and customer support into different systems. As a result, users must move across multiple screens and services to decide whether a product is worth buying. Traditional marketplaces also suffer from review manipulation, low-quality feedback, and poor product explainability.

BuzzCart is designed as a unified social commerce application where:

- sellers can publish products and content,
- buyers can discover products through media-rich feeds,
- users can interact through follows, reels, videos, and messages,
- reviews become more trustworthy through verified purchase checks and social ranking,
- product understanding improves through AI-driven document-based question answering.

The application therefore blends ecommerce, social networking, and AI assistance into one product.

3. Problem Statement

This project mainly solves two core problems:

3.1 Untrusted Product Reviews

In many ecommerce systems, reviews are shown as flat lists. A buyer often does not know:

- whether the review is from an actual purchaser,
- whether the reviewer is someone from their social circle,
- whether the review is public, moderated, or useful,
- which reviews should be trusted first.

This creates uncertainty and reduces confidence in purchase decisions.

3.2 Poor Product Understanding

Many products include long PDFs, brochures, manuals, or technical specification sheets. Users rarely read them fully, so they still have questions such as:

- What storage options are available?
- What battery life does this product offer?
- Does it support a specific feature?
- What are the ports, dimensions, or performance details?

Traditional search is inconvenient, and generic chatbots may hallucinate answers. Users need grounded, document-based product assistance.

4. Objectives

The main objectives of BuzzCart are:

- to build an integrated social commerce platform,
- to create a trustworthy review system based on verified purchase and social relevance,
- to provide a product-focused RAG assistant that answers only from indexed product documents,
- to support content-based discovery through posts, videos, and reels,
- to enable real-time buyer-seller communication,
- to support scalable deployment with containerized services.

5. Proposed Solution

BuzzCart proposes a multi-service platform with the following design:

- a Flutter client for web and app-style interaction,
- a Go backend exposing REST APIs and WebSocket messaging,
- PostgreSQL as the main system of record,
- Redis for temporary caching, especially ranked review results,
- Firebase Storage for product media, user media, avatars, and product documents,
- a Python RAG microservice using PDF parsing, sentence embeddings, FAISS indexing, lexical retrieval, reranking, and grounded answer generation.

The result is a system where product discovery, social proof, trust, and AI support are all connected.

6. System Architecture

6.1 High-Level Architecture

```
flowchart LR
    A[Flutter Frontend] --> B[Go Backend API]
    A --> C[Python RAG Chatbot]
    A --> D[WebSocket Messaging]
    B --> E[PostgreSQL]
    B --> F[Redis]
    B --> G[Firebase Storage]
    C --> E
    C --> F
    C --> H[FAISS Vector Store]
    C --> I[Embedding + Reranker Models]
    C --> G
    D --> B
```

6.2 Frontend Layer

The frontend is implemented in Flutter and uses:

- `provider` for state management,
- `go\_router` for navigation,
- a responsive layout with desktop sidebar and mobile bottom navigation,
- dedicated screens for home, shop, videos, reels, profile, cart, messages, upload, and settings.

It acts as the single user-facing interface for browsing products, interacting with content, managing the cart, writing reviews, and asking product-document questions.

6.3 Backend Layer

The backend is implemented in Go using the Gin framework. It provides:

- authentication and profile APIs,
- product and review APIs,
- cart and checkout APIs,
- feed and post APIs,
- video and reel APIs,
- follow and search APIs,
- upload endpoints for images, videos, avatars, and product documents,
- real-time messaging via WebSocket.

The backend also enforces important business rules such as purchase verification before review creation.

6.4 Database Layer

PostgreSQL stores the main transactional and social data, including:

- users,
- products and product images,
- product ratings,
- orders and order items,
- carts,
- content items and linked products,
- posts and feed tables,
- follows, conversations, and messages,
- moderation and helpful-vote metadata.

6.5 Storage Layer

Firebase Storage is used for:

- product images,
- user avatars,
- uploaded content media,
- product documents used by the RAG pipeline.

6.6 Cache Layer

Redis is used as a performance optimization layer. One important use case is caching ranked reviews for a short duration so repeated product page loads do not repeatedly recompute the same relationship-aware ordering.

6.7 AI RAG Layer

The chatbot service is a separate Python FastAPI application. Its main responsibilities are:

- downloading or accepting uploaded product documents,
- extracting PDF text,
- normalizing and chunking text,
- generating embeddings,
- storing product-specific FAISS indexes,
- performing hybrid dense plus lexical retrieval,
- reranking evidence with a cross-encoder,
- returning grounded answers with source page and chunk references.

This is not a generic chatbot. It is a strict product-document QA assistant.

7. Database Design Summary

The database design supports both ecommerce and social features.

Core ecommerce tables

- `products`
- `product\_images`
- `product\_ratings`
- `orders`
- `order\_items`
- `carts`

Core social tables

- `users`
- `user\_follows`
- `posts`
- `user\_feeds`
- `post\_likes`
- `post\_comments`
- `content\_items`
- `conversations`
- `messages`

Trust and moderation tables

- `review\_helpful\_votes`
- moderation fields on `product\_ratings`
- privacy fields on reviews and orders

This schema allows the system to connect real orders, public visibility, social relationships, and review ranking.

8. Main Modules and Features

8.1 Authentication and User Profiles

Users can register, log in, and manage their profile. The model supports different account roles such as seller, consumer, and admin. Privacy and visibility preferences are also supported, which affects whether purchases and social activity are visible to others.

8.2 Product Management

Sellers can create and manage products with:

- title,
- description,
- price,
- compare-at price,
- SKU,
- stock quantity,
- category,
- tags,
- media,
- metadata.

This makes the platform suitable for real catalog-based selling rather than just informal posting.

8.3 Shopping Cart and Checkout

The cart supports:

- adding items,
- updating quantities,
- removing items,
- clearing the cart,
- checking out.

During checkout, the backend:

- validates the logged-in user,
- loads cart items,
- verifies stock,
- creates an order and order items,
- reduces stock,
- clears the cart.

This creates real purchase records, which are later reused for verified reviews and buyer visibility.

8.4 Social Feed

BuzzCart includes an Instagram-style hybrid feed. It supports:

- follower-oriented feeds,
- discovery feeds,
- posts with photos or videos,
- engagement metrics,
- visibility and privacy-aware distribution,

- product rails mixed with content.

The home screen intentionally blends social content and purchasable products, creating stronger discovery than a traditional catalog.

8.5 Videos and Reels

The platform supports short-form and long-form content:

- videos,
- reels,
- comments,
- likes,
- product linking from content.

This enables content-driven commerce where users discover products from creator media instead of only searching a product list.

8.6 Messaging

The system includes direct user messaging with WebSocket support. This allows:

- real-time message delivery,
- conversation-based communication,
- typing events,
- product-related interactions between buyers and sellers.

This is a strong selling point because many shopping platforms still rely on delayed or external communication.

8.7 Search and Discovery

Search allows users to discover:

- products,
- videos,
- reels,
- users.

This supports both direct intent-based exploration and social discovery.

8.8 Media and Document Uploads

The backend supports uploads for:

- images,
- videos,

- user photos,
- avatars,
- product documents.

Product document upload is especially important because it powers the RAG assistant.

9. Core Innovation 1: Trusted Reviews From Known People

This is one of the strongest parts of the project.

9.1 Verified Purchase Enforcement

A user cannot review a product unless they have actually purchased it through a completed or delivered order. This directly reduces fake reviews.

9.2 Verified Purchase Badge

Each review stores whether it is a verified purchase, and the UI highlights this clearly. This increases buyer confidence.

9.3 Socially Ranked Reviews

When a logged-in user views product reviews, the backend does not simply sort by date. It uses social relationship weighting:

- direct follower of the current user gets highest priority,
- mutual follow gets high priority,
- followed users get next priority,
- unrelated public reviews appear later.

In practice, this means the system tries to show reviews from known people first. This is the meaning of your project's main trust idea: reviews are not just from strangers, but from people socially relevant to the buyer.

9.4 Helpful Votes

Users can mark reviews as helpful. Helpful reviews rise further in visibility, especially for unauthenticated users who do not have social ranking context.

9.5 Moderation and Privacy

Reviews support:

- moderation status,
- moderator note,
- private/public review visibility.

This gives the project stronger real-world quality control than a basic star-rating system.

9.6 Buyer Visibility

The product detail screen can show who bought the product, again preferring socially connected users first when available. This acts as additional trust evidence beyond just ratings.

10. Core Innovation 2: RAG for Better Product Understanding

The second major innovation is the product-document RAG assistant.

10.1 Why It Matters

Most users do not read long technical documents, but they still need precise answers before buying. A product assistant grounded in official documents solves this gap.

10.2 Document Pipeline

For each product document:

1. the PDF is uploaded or fetched from a URL,
2. the text is extracted and cleaned,
3. the text is chunked into manageable sections,
4. embeddings are generated,
5. a product-specific FAISS index is built,
6. metadata such as page number and chunk ID is stored,
7. future questions are answered only against that product's indexed document.

10.3 Retrieval Strategy

The chatbot uses more than one technique:

- dense retrieval using embeddings,
- lexical retrieval for keyword-based matching,
- neighbor expansion for better local context,
- reranking of chunks and sentences,
- grounded evidence selection.

This is stronger than a simple vector search and improves answer precision.

10.4 Grounded Answers

The chatbot returns:

- an answer,
- source page,
- source chunk,

- confidence label.

This makes the assistant explainable and reduces hallucination risk compared with free-form AI chat.

10.5 Product-Specific Scope

The system is deliberately scoped per product. This means:

- answers stay tied to the right product,
- indexes remain organized,
- unrelated documents do not pollute retrieval,
- users receive cleaner and more trustworthy product explanations.

10.6 Business Value

This feature improves:

- product clarity,
- customer confidence,
- pre-purchase decision making,
- support efficiency,
- product differentiation.

In short, the RAG assistant helps users understand a product before they buy it.

11. How the System Works for End Users

11.1 Buyer Workflow

1. The user logs in.
2. The user explores the home feed, videos, reels, or shop.
3. The user opens a product detail page.
4. The user sees price, media, reviews, buyer previews, and social proof.
5. The user can ask questions about the product document through the chatbot.
6. The user adds the item to cart and checks out.
7. After purchase, the user becomes eligible to leave a verified review.

11.2 Seller Workflow

1. The seller creates an account.
2. The seller adds a product with media and metadata.
3. The seller uploads product-related content and product documents.
4. The product becomes discoverable in the shop and social feeds.

5. Buyers engage, purchase, message, and review.

11.3 Review Workflow

1. A buyer purchases a product.
2. The system records the order and order items.
3. The buyer writes a review.
4. The backend verifies purchase history before allowing review creation.
5. The review is stored as a verified purchase review.
6. Other users see that review with trust signals and relationship-aware ranking.

11.4 Product QA Workflow

1. A document is uploaded for a product.
2. The chatbot indexes it.
3. A user asks a question from the frontend.
4. The chatbot retrieves evidence from the indexed document.
5. The system returns a grounded answer and confidence score.

12. Major Selling Points of the Project

BuzzCart has several strong selling points:

- It combines social media and ecommerce in one platform.
- Reviews are more trustworthy because only real buyers can review.
- Reviews are more useful because known or connected people are prioritized.
- Users can see social proof through buyer visibility and review previews.
- The product RAG assistant gives grounded answers from actual documents.
- The architecture is modular, service-oriented, and production-oriented.
- Real-time messaging improves buyer-seller interaction.
- The feed-based discovery model is more engaging than a plain catalog.

Among these, the two biggest differentiators are:

- trusted reviews from known people,
- RAG-based product understanding.

13. Technical Strengths

The project demonstrates several good engineering decisions:

- separation of frontend, backend, and AI service,
- PostgreSQL-backed transactional consistency,

- Redis caching for repeated expensive reads,
- product-specific vector indexing,
- clear route grouping and modular handlers,
- Docker-based deployment support,
- responsive frontend design,
- role, privacy, and visibility-aware business logic.

14. Limitations

Although the project is strong, some practical limitations remain:

- payment processing is simplified rather than fully integrated with a real gateway,
- recommendation logic is still mostly feed-based rather than deeply personalized,
- moderation workflows could be expanded with richer admin tooling,
- analytics and observability can be strengthened further,
- the RAG service is document-grounded, but product knowledge is only as good as the uploaded files.

These are good areas for future work.

15. Future Enhancements

Possible future improvements include:

- real payment gateway integration,
- seller dashboards and business analytics,
- recommendation models based on user behavior,
- multilingual RAG and multilingual UI,
- richer moderation dashboards,
- voice-based product assistant,
- deeper integration between reviews, buyer identities, and product reputation scoring,
- personalized AI suggestions based on user profile and purchase history.

16. Conclusion

BuzzCart is a strong example of a modern social commerce system built around trust and product understanding. Instead of treating reviews as anonymous star counts and documents as ignored attachments, the project turns both into active decision-support tools.

The review system improves trust by ensuring that feedback comes from verified buyers and by prioritizing people from the user's social network. The RAG subsystem improves product understanding by converting long product documents into searchable, answerable knowledge for end users. Together, these two ideas solve meaningful practical problems in ecommerce.

From an academic and engineering perspective, the project is valuable because it demonstrates:

- full-stack application design,
- database-driven social commerce workflows,
- trust-aware review ranking,
- AI-assisted product support using RAG,
- modular architecture suitable for real deployment.

BuzzCart is therefore not just an ecommerce project, but a trust-centered and intelligence-enabled commerce platform.